DOCUMENTO DE ESPECIFICAÇÃO E ARQUITETURA DE MACHINE LEARNING

Plano Diretor: Treinamento de LLM Especialista em Programação e Cibersegurança Defensiva

Estratégia de Fine-Tuning de Alta Precisão, Infraestrutura em Nuvem e Pipeline Completo de Curadoria de Dados

Hardware Dedicado	1x GPU 96 GB VRAM (RTX 6000 Ada / Blackwell Server) + 180 GB de Memória RAM
Camada de Armazenamento	VM Efêmera: ~193 GB livres em disco local | Persistência: 5 TB via Google Drive montado (FUSE)
Modelo Base Alvo	Qwen 2.5 Coder 32B Base / Instruct (Treinamento em BF16 nativo com LoRA de alta capacidade)
Objetivo do Sistema	Assistente técnico integral: desenvolvimento diário, auditoria SAST, execução em sandbox, tutoria socrática e orquestração MCP

1. Contexto Estratégico e Justificativa do Projeto

O desenvolvimento de um modelo de linguagem especializado na interseção entre engenharia de software e segurança ofensiva/defensiva busca preencher uma lacuna crítica dos modelos comerciais genéricos. Sistemas padrão de mercado frequentemente sofrem de dois problemas fundamentais: o *fenômeno de alucinação sintática* em cenários de baixo nível (Assembly, C, Rust) e o *bloqueio por alinhamento moral excessivo (False Refusals)*, onde tarefas legítimas de auditoria técnica (como explicar a mecânica de um buffer overflow, simular um payload para validação de sanitização em ambiente controlado ou auditar um arquivo binário) são sumariamente rejeitadas.

A infraestrutura disponível — dotada de **96 GB de VRAM** e **180 GB de RAM** — desbloqueia a capacidade de executar o fine-tuning de um modelo denso de **32 bilhões de parâmetros** diretamente em **Bfloat16 (BF16)** sem a necessidade de quantizações agressivas (4-bit / NF4). A quantização costuma introduzir ruído numérico nos tensores de projeção, degradando a capacidade do modelo de rastrear ponteiros, controlar fluxo de pilha e manter indentação rigorosa. Além disso, a ampla memória gráfica permite elevar a janela de contexto de treino para **16.384 tokens** com *FlashAttention-2*, permitindo que o modelo analise repositórios completos e códigos inteiros em vez de apenas funções isoladas.

2. Arquitetura Técnica do Pipeline de Treinamento

O treinamento será executado utilizando o framework *PEFT* com adaptadores **LoRA (Low-Rank Adaptation)** de alta capacidade sobre a base em precisão nativa. A formulação matemática garante que os pesos originais do modelo (W_0) permaneçam congelados, enquanto as matrizes de rank baixo (A e B) aprendem as novas sinapses de segurança:

$$W_{final} = W_0 + (\alpha / r) \cdot (B \times A), \text{ com } r = 64 \text{ e } \alpha = 128$$

Os adaptadores serão injetados em todos os módulos lineares das camadas de atenção e MLPs (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`). Com os pesos em BF16 ocupando aproximadamente 64 GB de VRAM, os cerca de 32 GB restantes são alocados para o otimizador *Paged AdamW*, gradientes e o cálculo dinâmico de ativações via *Gradient Checkpointing*.

> **Estratégia de I/O e Mitigação de Latência:** O Google Drive montado via FUSE apresenta alta latência para leitura contínua de milhares de pequenos arquivos. Para evitar a ociosidade dos Tensor Cores, o dataset consolidado em formato *Apache Parquet* será carregado diretamente nos **180 GB de RAM** do sistema (usando o diretório de memória compartilhada `/dev/shm`). Os checkpoints de validação periódicos serão gravados diretamente no Drive persistente de 5 TB a cada 250 passos, garantindo imunidade total contra desconexões temporárias da VM.

3. Plano Diretor de Aquisição e Curadoria de Datasets

Para garantir que o modelo adquira raciocínio defensivo aprofundado sem sofrer de *Catastrophic Forgetting* (esquecimento catastrófico de habilidades gerais de raciocínio lógico e sintaxe), o conjunto de dados final será rigorosamente balanceado em uma proporção estruturada totalizando aproximadamente **60.000 amostras instrucionais de alta densidade**.

Pilar do Dataset	Volume	Proporção	Objetivo Técnico e Fontes Primárias
Engenharia de Software Geral	15.000 amostras	25%	Preservar integridade de algoritmos, refatoração, documentação e Clean Code. Fontes: Subsets filtrados de Magicoder-Evol-Instruct e OpenHermes-2.5 Code.
Análise de Falhas e Patches (SAST)	20.000 amostras	33.3%	Identificação de vulnerabilidades (CWEs), corrupção de memória em C/C++, falhas lógicas e geração do código corrigido. Fontes: DiverseVul, Big-Vul, PrimeVul.
Engenharia de Detecção e Regras	10.000 amostras	16.7%	Criação e interpretação de regras de detecção defensiva. Fontes: Regras Semgrep AST, regras YARA para binários, assinaturas Sigma para SIEM e Bandit.
Raciocínio Investigativo (CoT)	15.000 amostras	25%	Raciocínio deliberado passo a passo (...) sobre exploração de memória, criptografia e descompilação. Fontes: Writeups técnicos de CTF e síntese via modelo de 70B.

3.1. Detalhamento das Fontes por Categoria

- **DiverseVul & PrimeVul:** Repositórios acadêmicos contendo dezenas de milhares de funções reais em C/C++ extraídas de projetos de código aberto. Cada entrada mapeia o código vulnerável antes do commit, a versão corrigida (patch defensivo), a mensagem explicativa e o identificador oficial do CWE (ex.: CWE-119, CWE-416).
- **GitHub Security Advisories (GHSA) & NVD Pipeline:** Mineração automatizada de repositórios públicos que receberam CVEs formais. O pipeline extrai o *git diff* estruturado: o contexto excluído alimenta o prompt do usuário como código a ser auditado; o contexto adicionado alimenta a resposta do assistente como a solução técnica definitiva.
- **SecCodePLT & OWASP Top 10 Data:** Coleção voltada a linguagens de alto nível (Python, Go, JavaScript, Rust), cobrindo vulnerabilidades web críticas: Server-Side Request Forgery (SSRF), Injeção de SQL/Comandos, desserialização insegura e controle de acesso quebrado (IDOR).
- **Corpus de Regras Semgrep & YARA:** Conjuntos de regras estáticas que treinam o modelo a formular expressões de matching estruturado (AST matching) para integrar scanners automatizados de pipelines CI/CD.

4. Pipeline de Processamento, Síntese e Validação de Dados

O pipeline de curadoria opera em quatro fases sequenciais executadas diretamente no notebook do Google Colab:

1. **Deduplicação e Limpeza Semântica:** Aplicação de hashing difuso (*MinHash LSH*) para eliminar amostras duplicadas entre as bases públicas e remoção de comentários ruidosos ou trechos incompletos que não compilam.
2. **Enriquecimento de Raciocínio (Teacher-Student):** Como os dados de CVEs públicos frequentemente possuem explicações sucintas, utiliza-se a própria GPU para instanciar temporariamente um modelo de raciocínio profundo (Llama-3.3-70B-Instruct ou DeepSeek-R1 em FP8 via *vLLM*). O modelo atua como "Professor", processando o código e gerando a cadeia explícita de reflexão interna dentro de delimitadores `<think> ... </think>`.
3. **Validação Sintática por Compiladores Reais:** Para garantir que o modelo não aprenda códigos quebrados, scripts Python injetam o código corrigido de cada amostra em compiladores e linters automatizados instalados na VM (`gcc -Wall -c`, `mypy`, `cargo check`). Amostras com falha sintática são descartadas.
4. **Consolidação em Formato Parquet:** Estruturação final das amostras no formato de Chat Template específico da arquitetura Qwen (`<|im_start|>system...<|im_end|>`).

4.1. Esquema de Organização no Armazenamento Persistente (5 TB)

```
      /content/drive/MyDrive/cyber_llm_project/ ├── raw_sources/ <- Arquivos
compactados de bases públicas (.tar.gz, .zip) | ├── diversevul_export.parquet | └──
ghsa_cve_commits.jsonl ├── synthesis_pipeline/ <- Scripts Python de extração, linter e
validação de AST | ├── validate_syntax.py | └── cot_generator.py ├── processed_data/ <-
Dados limpos categorizados por domínio | ├── general_coding_15k.parquet | ├──
sast_patches_20k.parquet | ├── detection_rules_10k.parquet | └──
cot_reasoning_15k.parquet ├── training_ready/ <- Dataset final particionado | ├──
train.parquet (57.000 amostras - 95%) | └── validation.parquet (3.000 amostras - 5%) └──
model_checkpoints/ <- Checkpoints LoRA salvos a cada 250 passos └──
qwen32b_cyber_lora_final/ (Pesos adaptados exportados prontos para merge)
```

5. Protocolo de Avaliação e Validação de Segurança Pós-Treino

Após o término do ciclo de treinamento e a fusão matemática (*merge*) dos adaptadores LoRA nos tensores principais do Qwen 2.5 Coder 32B, o modelo resultante será submetido a uma suíte de testes de validação técnica composta por três baterias:

- **Taxa de Recusas Falsas (False Refusal Benchmark):** Testes com 200 prompts técnicos contendo termos sensíveis (ex.: *"analisar vetor de injeção", "gerar shellcode para teste de laboratório em x86", "explicar fluxo de retorno em ROP chain"*). O modelo deve responder com análise técnica formal e zero recusas vazias.
- **Benchmark SecCodePLT / HumanEval:** Medição da taxa de aprovação funcional de código em Python, C e Rust para assegurar que a taxa de acerto de algoritmos permaneça superior a 75% na primeira tentativa (Pass@1).

- **Acurácia de Identificação de CWEs:** Apresentação de 100 trechos de código com vulnerabilidades reais nunca vistas pelo modelo para mensurar se a identificação da falha de memória ou lógica e a linha de código afetada atingem acurácia acima de 85%.

6. Ecossistema Operacional: Trabalho Diário, Ferramentas e Protocolo MCP

Para que o modelo transcenda a função de mero gerador textual e atue como um copiloto interativo na rotina profissional de engenharia e segurança, o servidor local em vLLM (porta 8000) é integrado a uma suíte de automação de ferramentas e editores de desenvolvimento.

6.1. Integração Direta com IDEs e Ambiente de Trabalho

- **Plugins de IDE (Continue.dev / Cline no VS Code e Neovim):** Conexão via endpoint compatível com a API da OpenAI. Permite preenchimento preditivo em tempo real através de *Fill-in-the-Middle (FIM)* com latência reduzida pelo Continuous Batching, além de chat contextual com leitura nativa dos arquivos e árvore do repositório ativo.
- **Agente de Terminal CLI (Aider / Shell Integrator):** Assistente em linha de comando capaz de interpretar pedidos de refatoração, mapear o histórico do Git, aplicar diffs cirúrgicos e redigir mensagens de commit padronizadas automaticamente.

6.2. Ambiente de Execução Segura (Sandboxed Code Interpreter)

O servidor incorpora um container isolado (Docker rootless / gVisor) conectado ao fluxo de inferência. Quando o usuário requisita validação de um algoritmo complexo em C, script de automação ou teste de regressão, o modelo submete o trecho ao ambiente estanque antes de emitir a resposta final:

- **Compilação e Checagem Estática:** Execução automatizada com flags rigorosas (`gcc -Wall -Wextra -Werror`, `cargo check`, `mypy`) para capturar ponteiros nulos ou incompatibilidades de tipo.
- **Inspeção Dinâmica de Memória:** Rastreamento de vazamentos de memória (*memory leaks*) e acessos fora dos limites via `valgrind` e AddressSanitizer (ASan), garantindo que apenas soluções verificadas sejam entregues ao desenvolvedor.

6.3. Conectividade via Model Context Protocol (MCP)

A adoção do padrão aberto MCP permite ao assistente consultar serviços do sistema sem necessidade de copiar e colar dados brutos:

- **MCP Filesystem & Git:** Inspeção direta de branches, histórico de diffs entre releases e navegação na árvore de diretórios local.
- **MCP Relacional (SQLite / PostgreSQL):** Inspeção de diagramas de tabelas, análise de índices e geração de planos de execução otimizados via `EXPLAIN ANALYZE`.
- **MCP Web Fetch:** Consulta em tempo real a documentações técnicas oficiais, RFCs recém-aprovadas e notas de versão de dependências externas.

6.4. RAG Técnico Híbrido (Retrieval-Augmented Generation)

Enquanto o fine-tuning consolida sintaxe e intuição de baixo nível, a precisão factual de manuais e protocolos extensos é garantida por um banco vetorial local (*Qdrant*). A base indexa: páginas de manual do kernel Linux (*man pages*), especificações completas de protocolos de rede (TCP, DNS, TLS), documentação da API de sockets BSD e catálogos de regras estáticas (regras Semgrep, YARA e assinaturas Sigma).

7. Plataforma de Estudos Técnicos, Tutoria Socrática e Memória Contínua

Para dar suporte à evolução técnica contínua e à preparação em tópicos avançados de computação e arquitetura defensiva, o assistente conta com módulos especializados de aprendizado:

Componente	Mecanismo Técnico	Impacto no Aprendizado
Memória de Longo Prazo	Camada de memória vetorial persistente (Mem0 / SQLite-vec)	Armazena histórico de tópicos já dominados, preferências de sintaxe e padrões de erros conceituais recorrentes.
Modo Tutor Socrático	Prompt de sistema instrucional parametrizado para dedução ativa	Evita respostas diretas; decompõe desafios de algoritmos, redes ou engenharia reversa em pistas e perguntas orientadas.
Gerador Anki (SRS)	Exportador automático em formato TSV estruturado com Cloze Deletion	Sintetiza cartões de repetição espaçada com comandos, conceitos de sistemas e RFCs ao final de cada sessão de estudo.

8. Arquitetura Integrada do Servidor Local e Fluxo de Execução

A infraestrutura local consolida a orquestração entre interfaces de desenvolvimento, camadas de execução protegida, bases de conhecimento técnico e o motor de inferência acelerado em VRAM:

